Speeding up Static Analysis with the Split Operator

Vincenzo Arceri, Greta Dolcetti, Enea Zaffanella
University of Parma



UNIVERSITÀ
DI PARMA

Presentation Outline

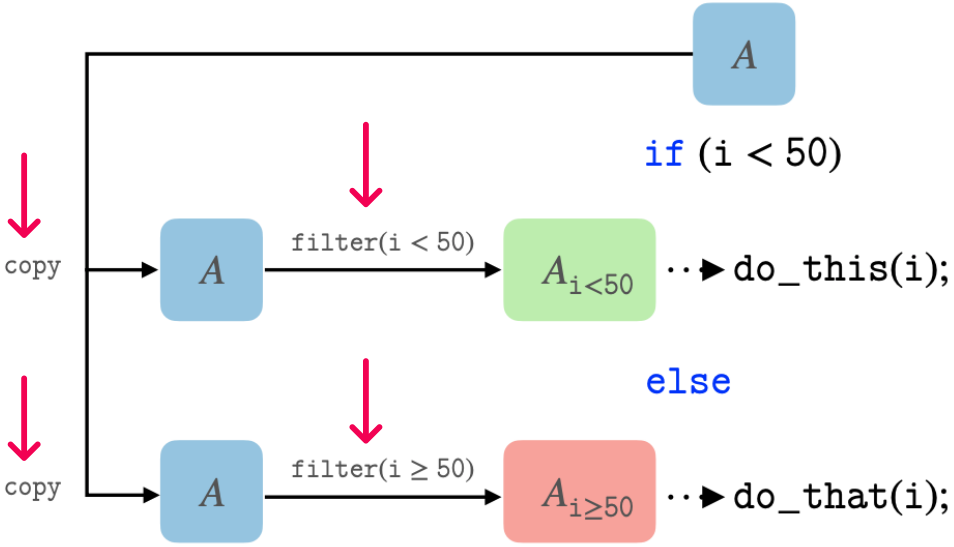
- Filtering an Abstract Element
- The Split Operator
  - Formalization
  - Examples
  - Implementation
- Experimental Evaluation
- Conclusion and Future Works

Filtering an Abstract Element

$$\mathrm{filter} : \mathbb{A} \times \mathrm{Pred} \rightarrow \mathbb{A}$$

- Common abstract operator
- Program control flow modeling
- Adopted by many tools (Crab, IKOS, LiSA, MOPSA, ...)

Example



- Correct and precise
- Inefficiencies
- Replicated work
- Overheads
- Two invocations

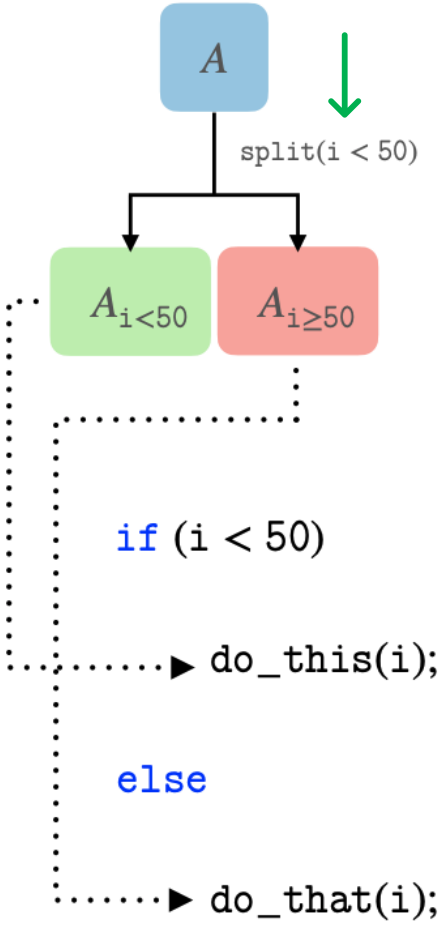
The Split Operator

$$\mathrm{split} : \mathbb{A} \times \mathrm{Pred} \to \mathbb{A} \times \mathbb{A}$$

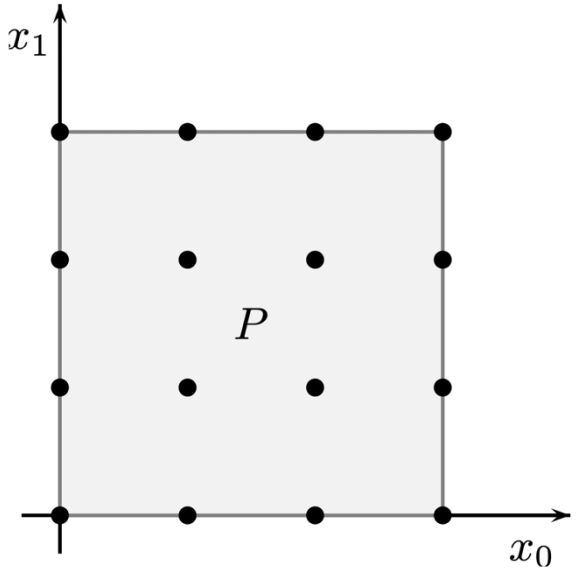
- Filter on both predicate and its complement *at the same time*
- Implemented in polyhedra library PPLite
- Goal: efficiency improvement

Example

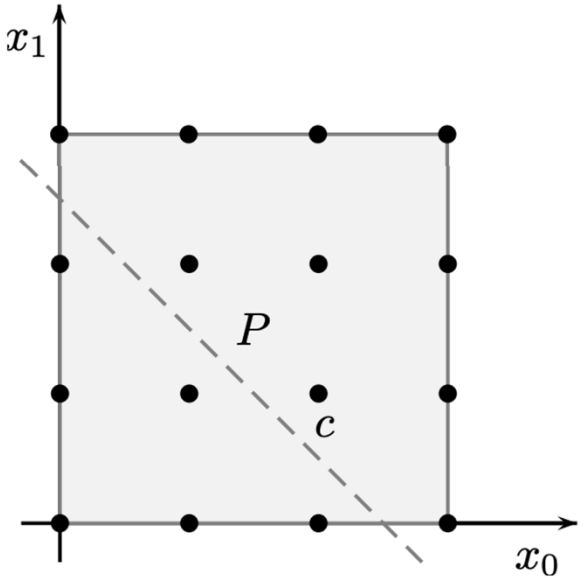
- Correct and precise
- No replicated work
- No overheads
- Single invocation



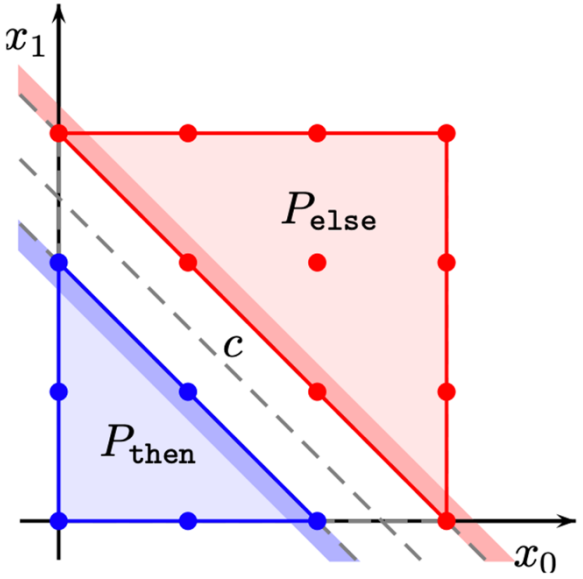
Splitting Polyhedra on Linear Constraints



$P \equiv \{0 \leq x_0 \leq 3, 0 \leq x_1 \leq 3\}$



Constraint $c \equiv (2x_0 + 2x_1 \leq 5)$

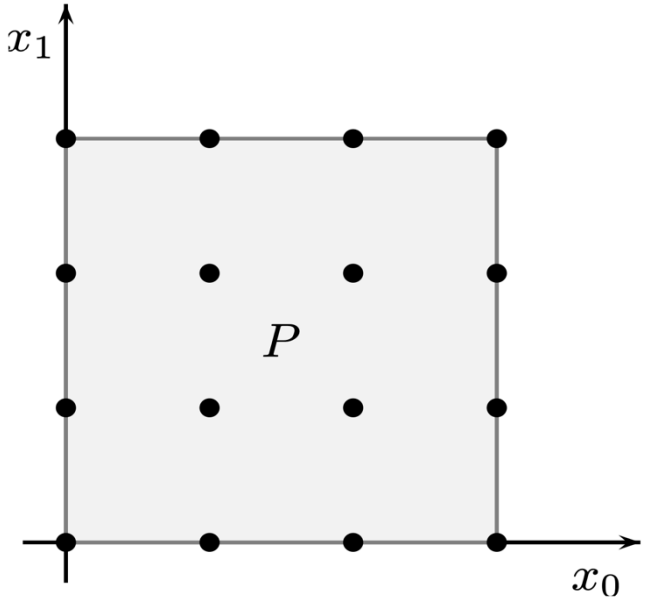


Split on $c \equiv (2x_0 + 2x_1 \leq 5)$

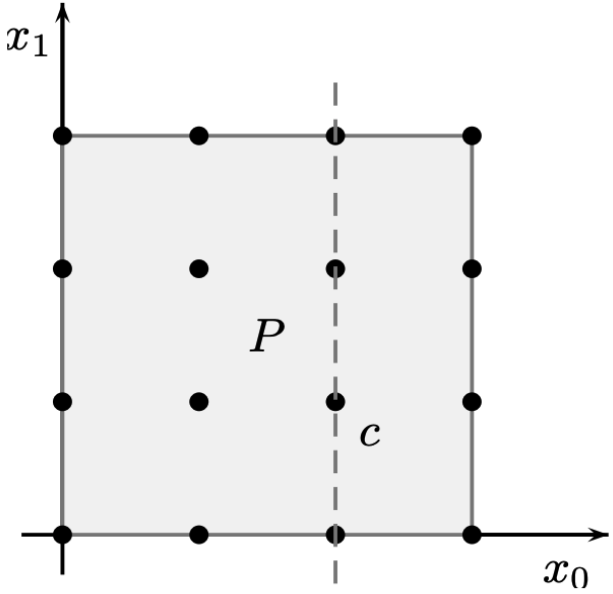
-> $x_0 + x_1 \leq 2$,

-> $x_0 + x_1 \geq 3$

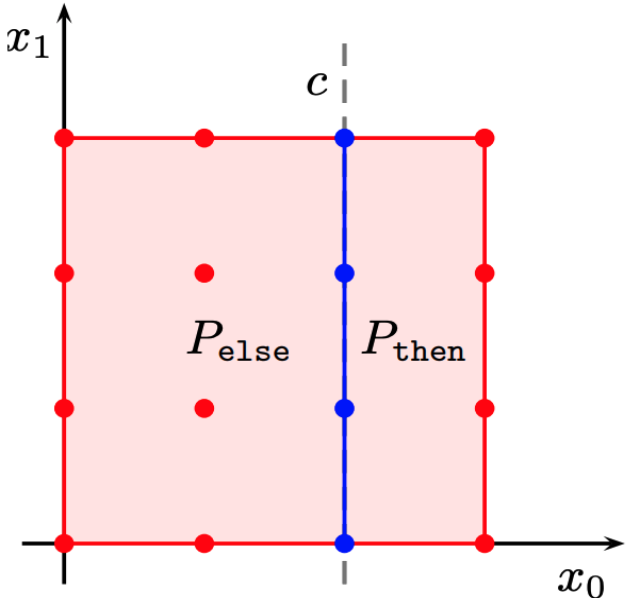
Splitting Polyhedra on Linear Constraints



$P \equiv \{0 \leq x_0 \leq 3, 0 \leq x_1 \leq 3\}$

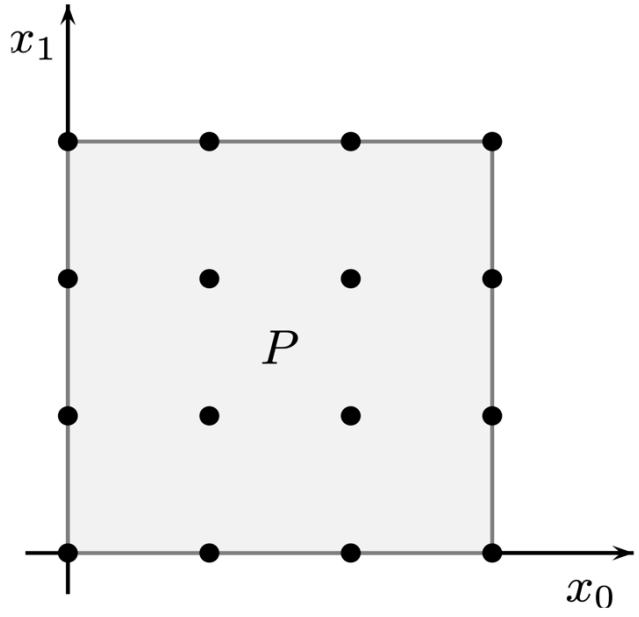


Constraint $c \equiv (x_0 = 2)$

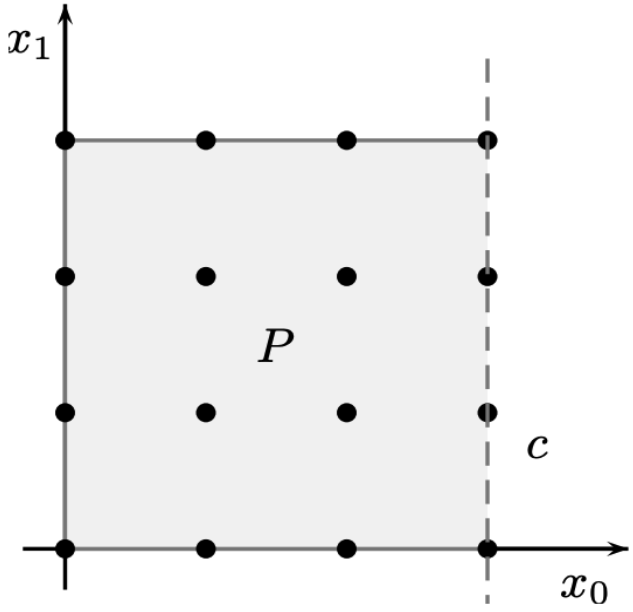


Split on $c \equiv (x_0 = 2)$

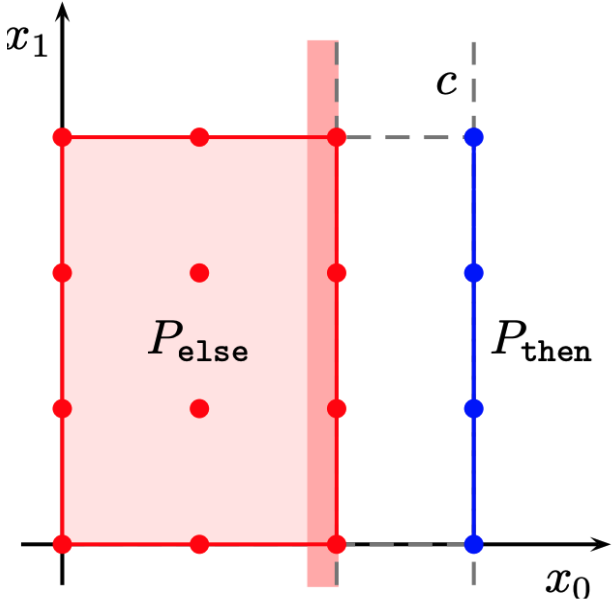
Splitting Polyhedra on Linear Constraints



$P \equiv \{0 \le x_0 \le 3, 0 \le x_1 \le 3\}$

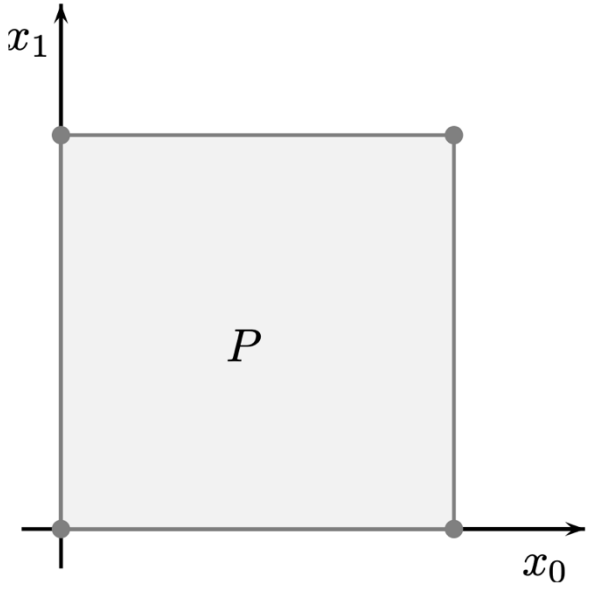


Constraint $c \equiv (x_0 = 3)$



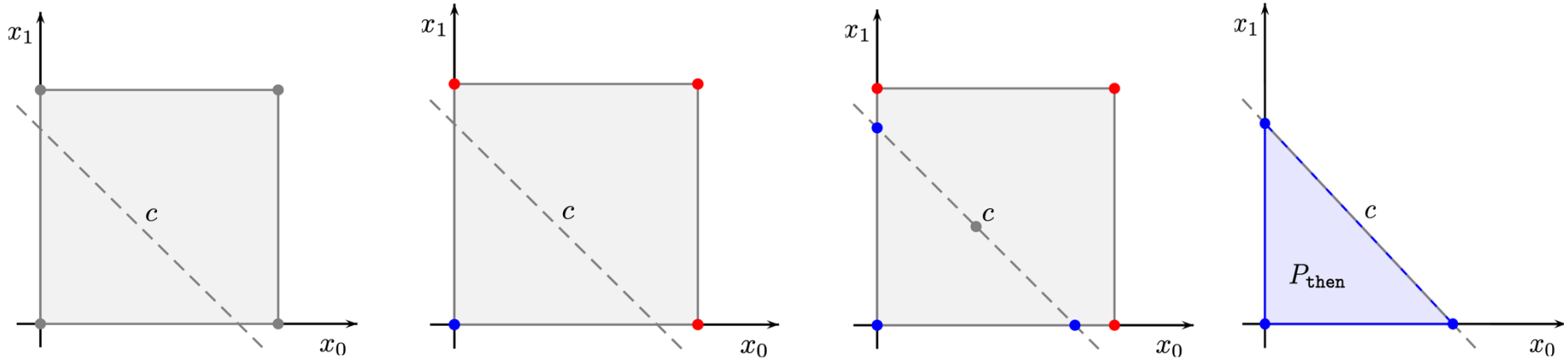
Split on $c \equiv (x_0 = 3)$

Splitting Polyhedra on Linear Constraints



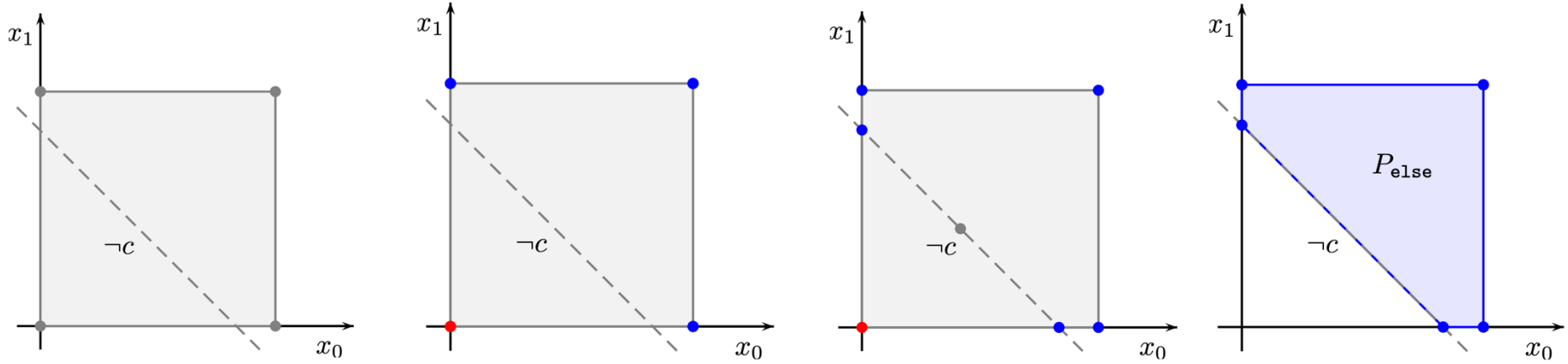
$P \equiv \{0 \leq x_0 \leq 3, 0 \leq x_1 \leq 3\}$

Splitting Polyhedra on Linear Constraints



$c \equiv (2x_0 + 2x_1 \leq 5) \rightarrow P_{then} = \text{filter}(P, c)$

Splitting Polyhedra on Linear Constraints



$\neg c \equiv (2x_0 + 2x_1 \geq 5) \rightarrow P_{then} = filter(P, \neg c)$

Clam/Crab

- Modules of the Seahorn framework
- Crab
  - C++ library for static analysis
  - Abstract domain and fixpoint engines
  - CrabIR
- Clam
  - Frontend for Crab
  - C → LLVM → CrabIR

Implementation in Crab

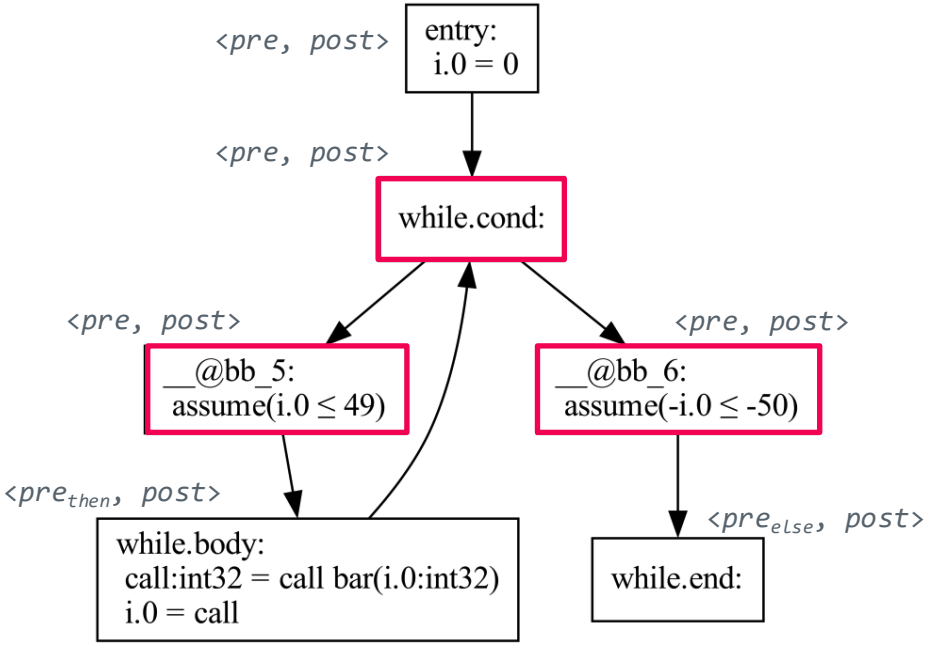
- Changes in the abstract domain component
  - New method for the split operator
  - Call to ad-hoc function of PPLite library
  - Handling of unoptimized default implementations

Implementation in Crab

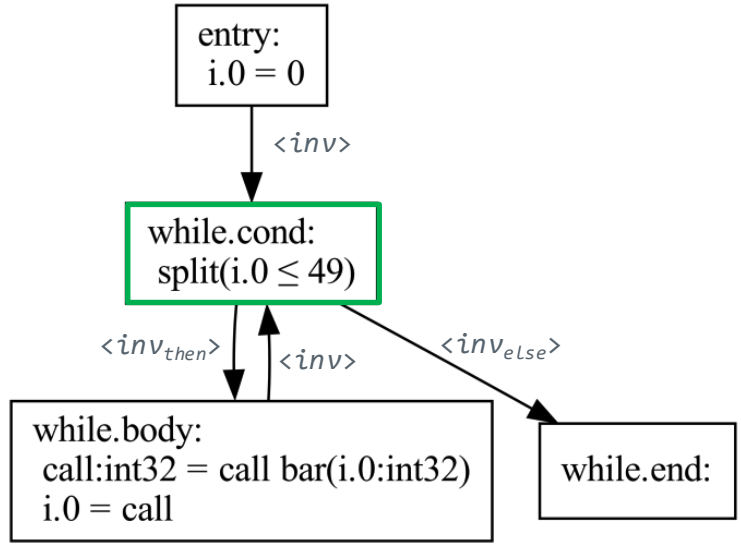
- Changes in the fixpoint approximation engine
  - New abstract statement (`split`)
  - Invariants stored on edges

Example

```
void foo() {int i=0; while(i<50) i=bar(i);}
```



Default CFG



CFG with `split` stmt

Notes

- In Crab some of the tests are ignored:
  - Floating point
  - Unsigned variables
- Our prototype also ignores `select` statements

Experimental Evaluation

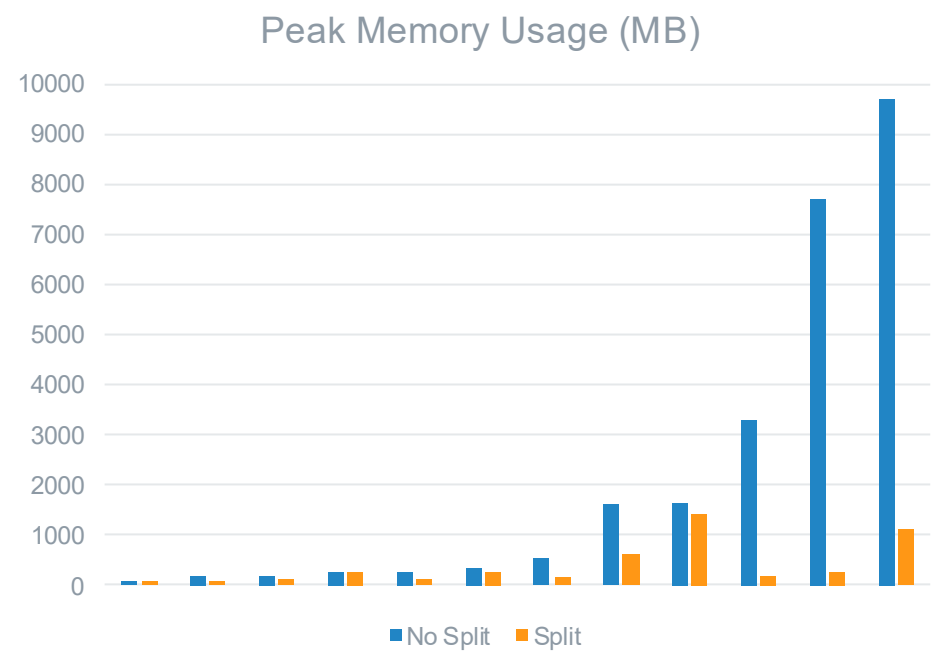
- PPLite and ELINA w/o split and PPLite with split
- Two benchmarks:
  - WCET analysis (PAGAI, 39 files)
  - Linux drivers (SVCOMP, 7 files)
- Crab configuration:
  - `switch` statement are lowered
  - Systematic inlining of function calls

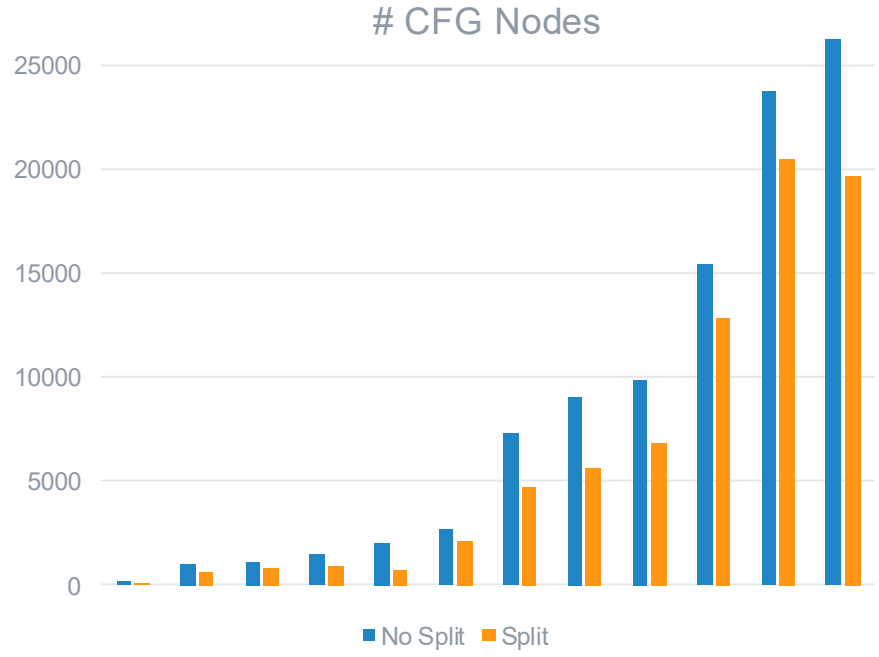
Experimental Evaluation - Times

(abridged) test name	without splits PPLite	without splits ELINA	with splits PPLite	time ratio
adpcm	108.2	(★) 1.6	105.9	1.02
prog9000	31.8	241.1	42.0	0.76
nsichneu	30.9	40.8	21.1	1.46
decompress	5.3	5.5	2.3	2.30
filter	2.5	2.3	2.5	1.00
mmc-host	487.1	435.1	412.9	1.18
firewire	98.7	(★) 92.0	22.7	4.35
hwmon-abituguru3	87.5	91.0	52.8	1.66
media-usb-tm6000	23.2	22.3	12.4	1.87
media-pci-ttpci	12.3	12.9	10.8	1.14
power-bq2415x	10.1	10.9	11.6	0.87
hid-usb	8.6	10.1	6.0	1.43

- Usually significant speedup
- A few slowdowns (due to the finalization phase)

Experimental Evaluation - Memory





Conclusion

- Results
  - Prototypal implementation
  - Non trivial memory and time improvements
  - No precision losses
- Future works
  - Split operator for other abstract domains
  - Extension to other tools

Thanks!

Speeding up Static Analysis with the Split Operator

Vincenzo Arceri, Greta Dolcetti, Enea Zaffanella
University of Parma

